

Derivation of the KSB Architecture

1. Introduction

This document derives the architecture of the KSB from first principles. The premise is minimal: a stream of inputs arrives at variable pitch δp_i from an upstream source and must be handed off to a downstream carrier that expects them at fixed intervals. From this premise the buffer, its segmentation, and the downstream rate surplus are each shown to be necessary by characterizing what fails without them. The derivation arrives at a system with three levers for managing the residual failures — conditioning control π^C shaping the entry boundary conditions, the downstream rate and slot parameters (η_r, d_s, v_d) , and the buffer geometry $(N^B$ and the segment lengths $L_k^B)$ — and characterizes how each acts on the failure modes that remain.

The derivation language throughout is the kinematic margin $M_{i,k}$ of *Buffer Formalization* §2.2 and the two failure modes it unifies. Each chapter introduces a single failure mode by exhibiting the condition under which it appears, and the chapters chain: the residual failure left by one becomes the subject of the next. The chain is logical, not historical — not the order in which the design unfolded, but the order in which the role of each piece is most cleanly established. The document divides into two registers along this chain. In the first, each failure motivates a subsystem that resolves it: mechanical decoupling motivates the buffer, the single-actuator limit motivates segmentation, unbounded drift motivates the rate surplus. In the second, the failures that remain — the concentration of both failure types at the entry and exit edges, and the periodic pressure tied to skip events — are not resolved here. They are characterized as precisely as the margin allows, the constraints any solution must satisfy are stated, and candidate directions are named. Presenting these as open is deliberate: a sharply posed open problem is a stronger result than an overstated resolution.

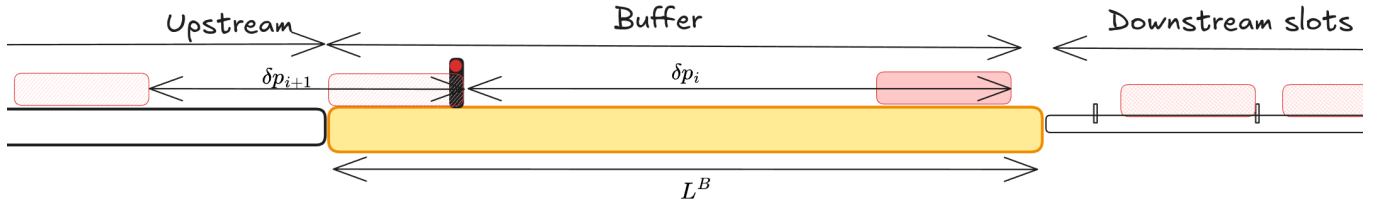
2. The Buffer

The upstream delivers inputs at variable pitch — a stochastic process with mean μ_u and standard deviation σ_u . The downstream carrier runs at constant velocity v_d and presents a fixed sequence of slots at spacing d_s , each passing the handoff boundary at a precise, predetermined time. The two sides are mechanically decoupled and operate at independent rates. Left unmediated, an input arriving slightly early or late relative to the slot grid has no mechanism to correct for the offset — it either misses its slot or collides with the one ahead. A synchronization element is needed between them.

The classical solution is a servo-driven conveyor belt of length L^B inserted between the two stages. The belt can modulate its velocity independently of both upstream and downstream,

and by shaping the velocity profile an input experiences during its transit, it can adjust the input's arrival time at the handoff boundary to coincide with its assigned slot. This is the buffer.

For each input, a trajectory is generated by a policy π^I that maps the input's entry state, its assigned slot, and the available time horizon to a commanded velocity profile over the buffer length. The choice of trajectory family is a design decision: common options include 7-phase jerk-limited S-curves and 5th-order polynomials. Throughout this document we assume jerk-limited trajectories, as these respect the physical bounds on acceleration rate imposed by the belt mechanics and the contact between belt and input.



The buffer's length L^B is set by the kinematic requirements of the system and is in any realistic configuration much larger than the upstream input pitch: $L^B \gg \mu_u$. This means multiple inputs are simultaneously in transit on the buffer at any given moment. The buffer is a single belt: its velocity is transferred equally to every input it carries. Each input has been assigned a distinct downstream slot and therefore requires a distinct velocity trajectory — but π^I can only command one velocity signal at a time, and that signal reaches all co-resident inputs simultaneously. The kinematic constraints that define the trajectory family do not change this: no matter how π^I is parameterized, it cannot resolve the fact that a single actuator is being asked to independently control multiple loads at once.

3. Segmentation

The traditional solution is to divide the buffer into independently controlled segments, reducing the likelihood that any single segment carries more than one input at a time. Each segment has its own servo drive and can transfer kinematics to whatever input it currently bears, independent of the other segments.

The segments cooperate to execute a single smooth trajectory for each input — for instance a 7-phase jerk-limited S-curve — with each segment responsible for a contiguous portion of that profile. As input i crosses the boundary into segment k at time $t_{i,k}^{\text{in}}$, segment k synchronizes its velocity and acceleration to match the incoming kinematic state, ensuring a smooth handoff with no discontinuity in the input's motion.

This synchronization is not instantaneous. Before input $i + 1$ arrives, segment k must transition from the state it held at the end of input i 's transit to the state it will need at the start of input $i + 1$'s transit. This transition requires a minimum time $M_{i,k}$, computed from the boundary states and the kinematic bounds of the synchronization policy π^S . The actual time available for this

transition — the window during which segment k carries neither input i nor input $i + 1$ — is the free window $W_{i,k}$. Feasibility requires $W_{i,k} \geq M_{i,k}$.

The policy π^S operates under strictly higher kinematic bounds than π^I , because during the free window no input is present on the segment and the no-slip constraint does not bind. π^S is limited only by the servo's own envelope rather than by the friction at the belt-input interface. This bound separation is what makes the free window an exploitable resource: the segment can execute the required synchronization transition in time precisely because it has more kinematic authority when it is empty than when it is bearing an input.

3. The Two Failure Modes

The segmentation chapter established two conditions a segment must satisfy: the free window must exist ($W_{i,k} \geq 0$), and it must be long enough for π^S to complete the synchronization transition ($W_{i,k} \geq M_{i,k}$). Violating either condition produces a distinct failure.

When $W_{i,k} < 0$, input $i + 1$ arrives at segment k before input i has cleared it. Two inputs are simultaneously on the same segment, which can only command a single velocity — the situation the segmentation was introduced to prevent. This is a **co-occupancy failure**.

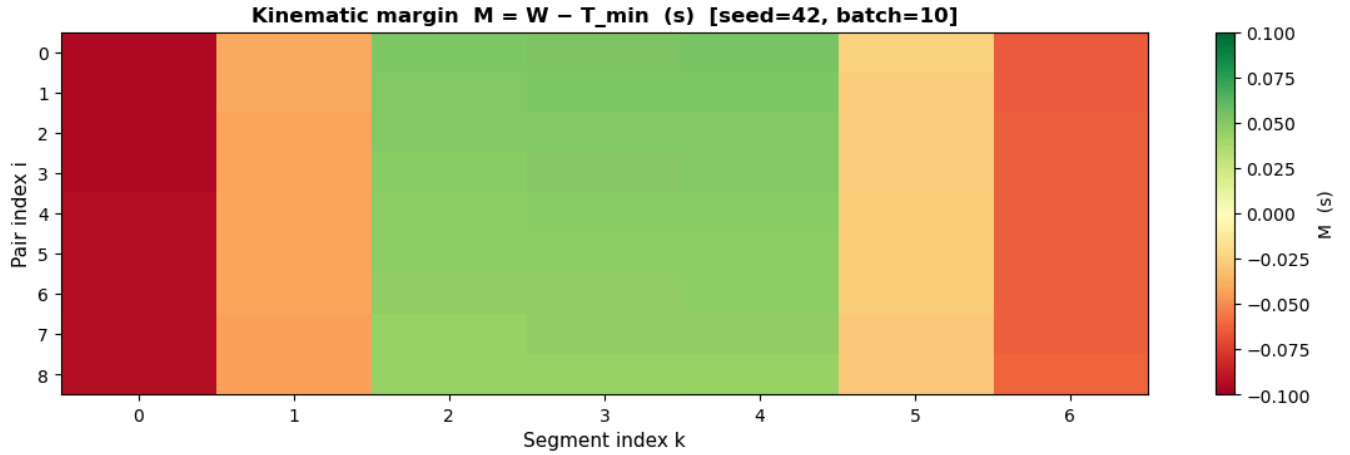
When $W_{i,k} \geq 0$ but $W_{i,k} < M_{i,k}$, the segment is empty long enough for a transition to begin, but not long enough for π^S to complete it within the available window. The segment arrives at $t_{i+1,k}^{\text{in}}$ in the wrong kinematic state, and the smooth handoff is broken. This is a **synchronization failure**.

The signed quantity $M_{i,k} := W_{i,k} - T_{\min}(\zeta_{i,k})$ — the kinematic margin — unifies both: $M_{i,k} \geq 0$ is feasible, $M_{i,k} < 0$ is not. Co-occupancy corresponds to the regime where $W_{i,k}$ is itself negative; synchronization failure to the regime where $W_{i,k}$ is positive but insufficient. The two are distinguished by the sign of $W_{i,k}$ alone, without evaluating $M_{i,k}$. Here $\zeta_{i,k}$ denotes the transition π^S must execute on segment k between inputs i and $i + 1$ — fully characterized by the kinematic state at which segment k exits input i and the state it must reach before input $i + 1$ arrives. $T_{\min}(\zeta_{i,k})$ is the minimum time π^S requires to execute that transition under its kinematic bounds.

4. Analyzing a Design

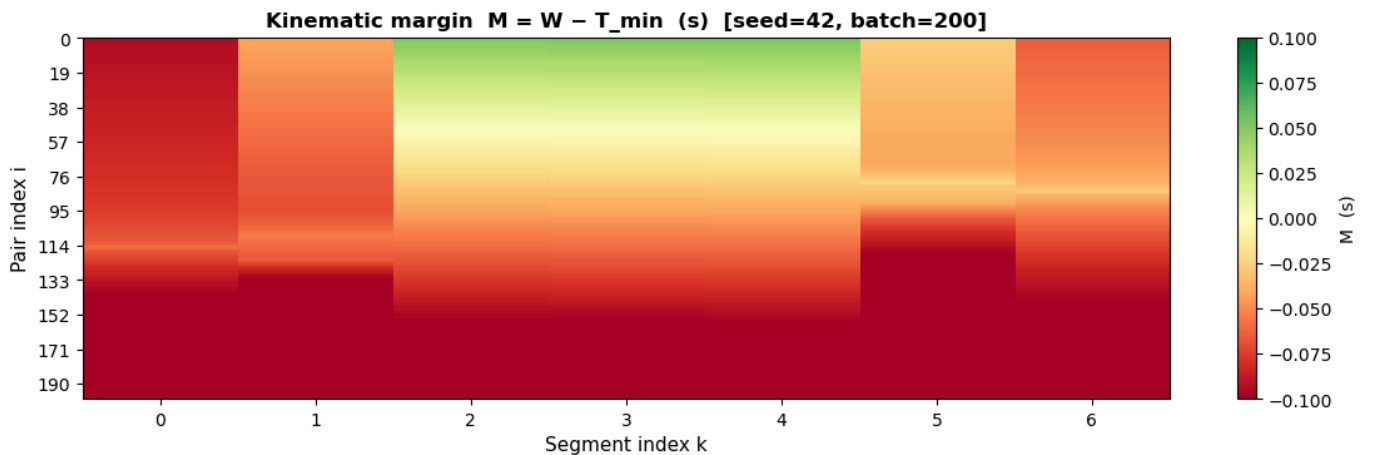
With $M_{i,k}$ defined for every event pair (i, k) , we can evaluate a specific buffer design by running it over a batch of b inputs and inspecting the resulting margin matrix. Each row corresponds to a consecutive input pair i , each column to a segment k . A positive entry means the segment had enough free window to complete its synchronization transition; a negative entry means it did not.

Consider a buffer with $N^B = 7$ equally sized segments, total length $L^B = 2.5$ m, upstream mean pitch $\mu_u = 0.6$ m and downstream pitch $d_s = 0.64$ m. For a batch of $b = 10$ inputs the matrix is small enough to read directly. Green entries are feasible, red entries are not — the color encodes the signed margin $M_{i,k}$, with white at zero.



The pattern is immediate: the interior segments (roughly $k = 2$ through $k = 4$) are uniformly healthy across all pairs, while the two outermost segments on each side ($k = 0$, $k = 1$ and $k = 5$, $k = 6$) show degraded or negative margins. This is the **edge concentration** effect. The interior segments have both boundary conditions negotiated with neighboring segments — they absorb disturbances incrementally and are insulated from the outside world. The edge segments do not have this insulation. Segment $k = 0$ inherits whatever kinematic state the upstream stage delivers, with no opportunity to shape it. Segment $k = 6$ must hand off at precisely the state the downstream stage demands, regardless of what trajectory the buffer executed to get there. One boundary condition is externally imposed at each edge, and the margin deficit is the cost of that constraint. This asymmetry is structural and present even in short batches.

Now extend the same configuration to $b = 200$ inputs.



A second problem appears, independent of the edge effect: the kinematic margin decreases steadily with batch position across all segments. The interior segments start healthy but deteriorate as i increases, eventually crossing into negative territory. The edge segments,

already degraded at $b = 10$, go further negative. The drift is uniform across k — all segments shift together — and it grows without bound as the batch lengthens.

This is a different kind of problem from edge concentration, and it requires separate treatment. The drift is not caused by the boundary asymmetry at the edges; it is caused by something in the relationship between the upstream arrival process and the downstream slot rate. Its structure has a direct parallel in queuing theory, where the same accumulation phenomenon arises when a server operates at or near its capacity limit. Understanding why the drift is unbounded — and what condition on the rates removes it — is the subject of the next chapter.

5. The Drift Problem

The margin heatmap showed that for long batches the kinematic margin decreases uniformly across all segments. This is not an edge effect — it is a property of the system as a whole. To understand it we need to look at the relationship between the upstream arrival rate and the downstream slot rate.

Define the upstream arrival rate $r_u = v_u/\mu_u$ and the downstream slot rate $r_d = v_d/d_s$. The key quantity is the time horizon H_i — the gap between when input i arrives at the buffer and when its assigned slot is ready. As H_i increases, the buffer has more time to execute its trajectory, and the kinematic margins improve. As H_i decreases, the margins tighten. The drift in the heatmap is directly the drift in H_i .

Each step, H_i shifts by the difference between the inter-slot time and the inter-arrival time:

$$B_i = \frac{d_s}{v_d} - \frac{\mu_u + \epsilon_i}{v_u}$$

When $r_d = r_u$ the deterministic parts cancel and $B_i = \epsilon_i/v_u$, with $E[B_i] = 0$. The noise is symmetric — positive and negative steps are equally likely. One might expect the process to fluctuate around a fixed mean. It does not.

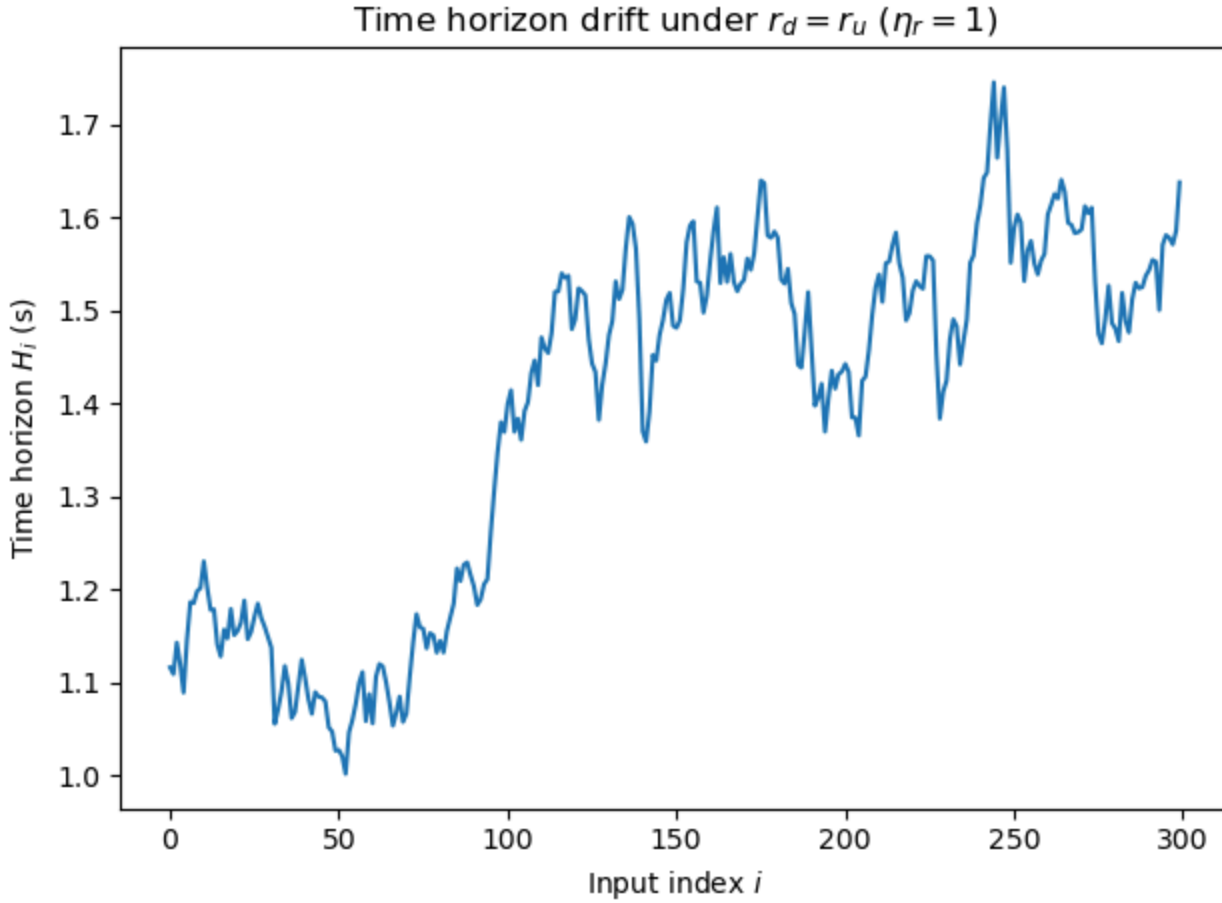
The reason is the floor. π^I requires a minimum time $T_{\min}$ to execute the buffer transit, and each slot can hold only one input. When $H_i + B_i$ would drop below $T_{\min}$, the slot is infeasible and the greedy assignment jumps to the next one. The recursion is therefore:

$$H_{i+1} = \max(T_{\min}, H_i + B_i)$$

with $H_1 > T_{\min}$ given. The floor makes the consequences of positive and negative steps asymmetric: positive steps always carry forward in full, while negative steps are truncated whenever they would push H_i below $T_{\min}$. Therefore $E[H_{i+1}] \geq E[H_i]$ at every step, with strict inequality whenever $P(H_i + B_i < T_{\min}) > 0$.

This is a well-known result. The recursion above is the **Lindley recursion** from queueing theory, and the regime $r_d = r_u$ corresponds to a queue at critical load. At critical load the

process is null-recurrent: it has no stationary distribution and $E[H_i]$ grows as $O(\sqrt{i})$ [Lindley, 1952]. The plot below confirms this — the time horizon drifts upward without bound over a long batch, even under symmetric i.i.d. noise.

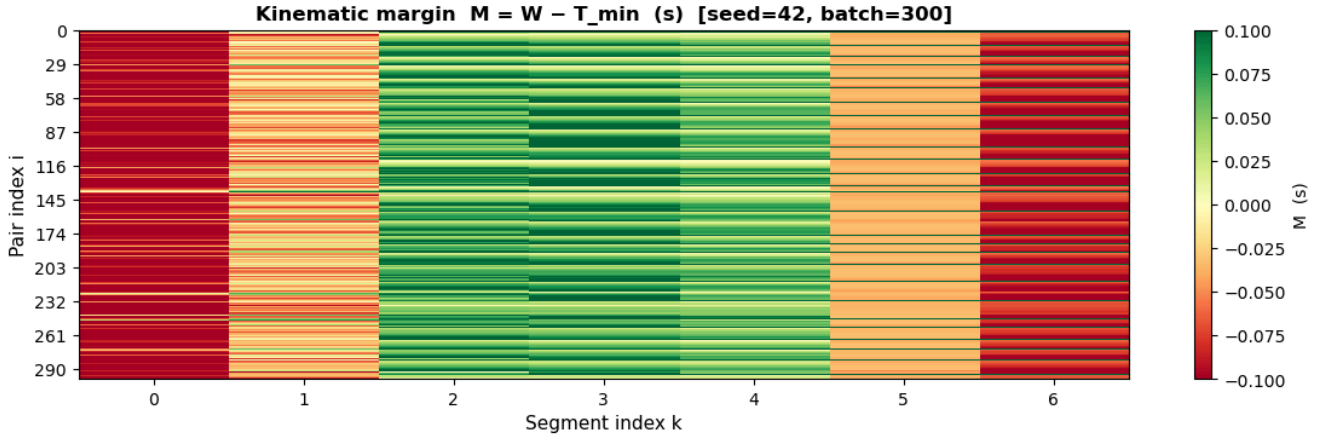


The solution is to introduce a rate imbalance. The recursion needs a negative drift term to counteract the asymmetry of the floor — something that systematically pulls H_i back down. In principle there are two ways to achieve this: decrease d_s so that slots arrive more frequently, or increase v_d so that the downstream runs faster. Both amount to making $r_d > r_u$. Decreasing d_s , however, tightens the spacing between downstream slots and directly worsens the edge concentration problem at segment N^B — the buffer has less time to decelerate each input before handoff. The natural lever is therefore r_d : run the downstream carrier slightly faster than the upstream, so that $r_d > r_u$ and the recursion acquires a negative drift. The next chapter introduces this rate slack and shows that it is sufficient to bound $E[H_i]$.

The natural lever is therefore r_d : run the downstream carrier slightly faster than the upstream, so that $r_d > r_u$ and the recursion acquires a negative drift. This is a well-known heuristic in industrial automation. The standard choice is $r_d = \eta_r r_u$ with $\eta_r = 1.1$ — a 10% rate surplus. This is a heuristic, and an optimal η_r may well exist for a given σ_u and pitch error distribution. Since we also need to address the edge concentration problem, we will be working with $\eta_r > 1$ throughout the remainder of this document.

6. Skip

Introducing $\eta_r > 1$ bounds the global drift, but it does not eliminate pressure on the buffer entirely. The heatmap below shows the margin matrix under $\eta_r = 1.1$ for a long batch.



The uniform downward drift is gone, but a new structure appears: the kinematic margin decreases in phases and then resets. The pattern repeats across the batch with a characteristic period. This is no longer a global accumulation problem — it is a local one, periodic in nature.

The source of this structure is the rate surplus itself. With $r_d > r_u$, the downstream presents slots faster than inputs arrive. Not every slot can be filled. At some point the greedy assignment cannot place an input in the next available slot and must skip it, jumping to a later one. These **skip events** are the direct consequence of $\eta_r > 1$, and they reset the phase relationship between the input stream and the slot grid.

7. The Exit Edge

The margin field is not uniform across segments. Measured as the failure rate $P(M_{i,k} < 0)$ per segment over a batch, the deficit concentrates at the boundaries: the interior segments carry comfortable margins while the segments near each edge run close to, or below, zero. The two edges fail for different reasons and admit different fixes. This chapter treats the downstream edge — the segments approaching $k = N^B$ — because its deficit has the simplest structure and the simplest lever. The upstream edge is taken up separately.

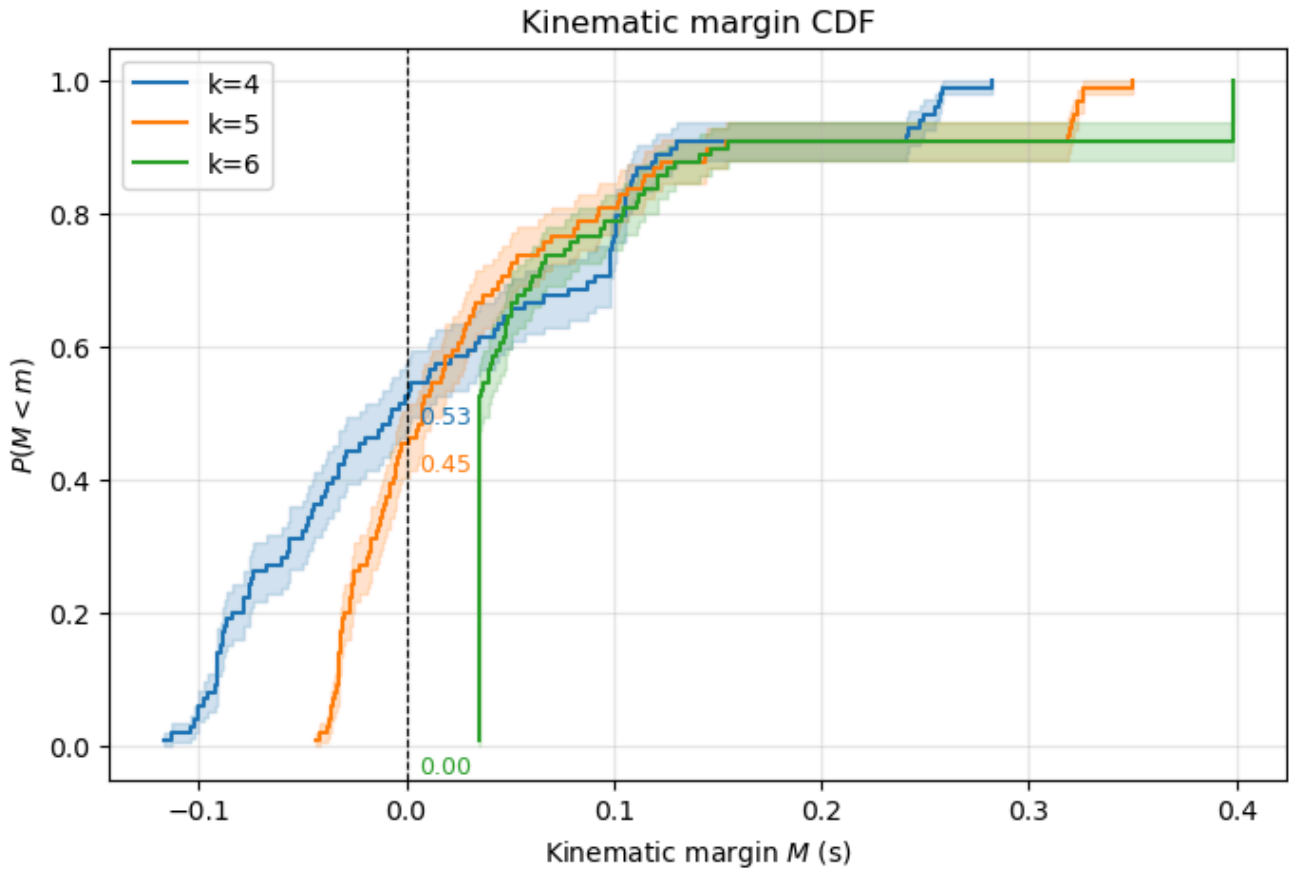
The downstream edge is defined by a boundary condition it does not get to choose. Every input must arrive at segment N^B in the kinematic state the downstream carrier demands: travelling at v_d , ready to be deposited into its slot. This state is fixed, identical for every input, imposed from outside the buffer. The interior segments negotiate both of their boundary conditions with their neighbours and can absorb a disturbance by spreading it across the transition; the downstream edge cannot. It has one end pinned to a constant, and whatever margin deficit that pinning produces is paid at the same segments on every input. The deficit is therefore deterministic and structural rather than noise-driven — it does not whip around with the upstream disturbance the way the entry edge does. It sits as a steady pressure on the segments that must bleed off the buffer's transit velocity and settle to v_d in time for the handoff.

The lever follows directly from what the deficit is. The segments near N^B are short of time to complete their transition into the pinned state. Time is the resource, and there are two independent ways to supply more of it. One is to give those segments a longer free window by changing the buffer geometry — making the end segments shorter so that each carries a smaller share of the deceleration. The other is to change the demand itself by widening the spacing between outgoing inputs: if consecutive inputs are further apart at the handoff, each downstream-edge transition has more time before the next input arrives. The optimal design is almost certainly a combination of the two, and the geometry lever is deferred to a later chapter. Here we take the spacing lever, because it requires no change to the buffer at all — only to the downstream parameters.

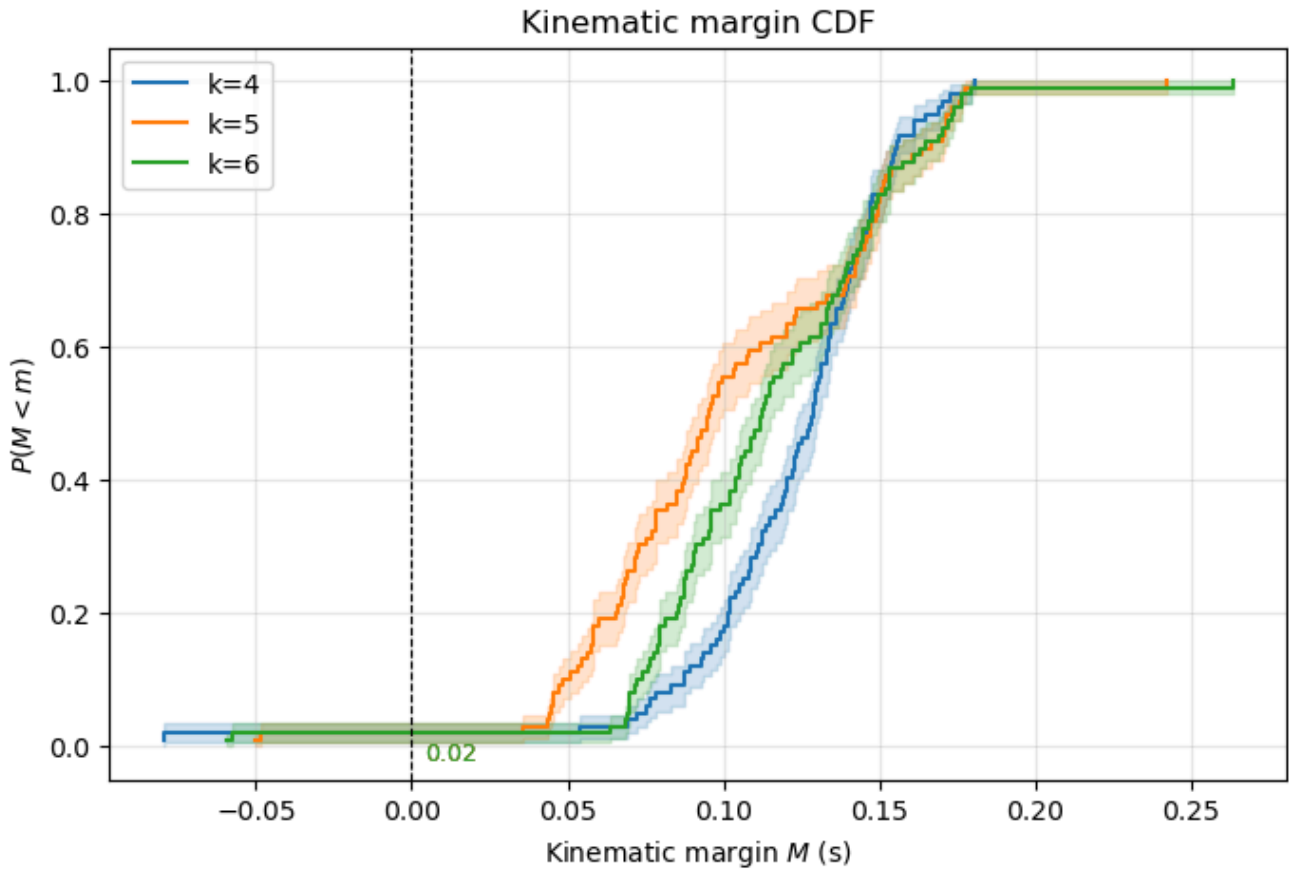
The spacing between outgoing inputs is set by the downstream slot schedule, and the schedule has so far been fixed by a single heuristic: the rate surplus $\eta_r = 1.1$. That number was introduced only to give the drift recursion a negative drift; its precise value was never derived, and nothing constrains it to 1.1. Widening the outgoing spacing means increasing the slot rate r_d relative to the arrival rate r_u — that is, increasing η_r .

There are two ways to realise a higher $r_d = v_d/d_s$: enlarge the slot size d_s , or present more slots per unit time. The downstream velocity is set by both, $v_d = r_d d_s$, so the choice between them is not cosmetic — it determines what else moves when the spacing widens.

Enlarging d_s widens the spacing, but at a cost that surfaces at exactly the wrong moment. The number of slots presented per batch is unchanged; the schedule simply spreads the same slots further apart. So when a skip occurs, the input must wait for a slot that is now further away in time, and the kinematic jump imposed by the skip is larger. Enlarging the slot trades a steady improvement in the downstream-edge margin for a sharper penalty on every skip. The figure below shows the result of this lever alone, at $d_s = 3.9l$: the outgoing spacing is wider, but the downstream-edge segments still straddle the failure line — $k = 4$ and $k = 5$ fail on roughly half of all inputs.



Increasing the slot *rate* avoids this. Presenting more slots per unit time, at unchanged slot size, widens the average outgoing spacing without lengthening the gap a skip must bridge: there are simply more slots available, so the next one after a skip is closer. This is the cleaner lever, and it is bounded by a single physical constraint. A higher slot rate raises $v_d = r_d d_s$, and the buffer must still be able to reach that handoff velocity, so $v_d \leq V_{\max}$. Within that bound, increasing the slot count is the simplest available improvement to the downstream edge. The figure below shows the effect of taking $r_d = 2 r_u$ with $\eta_s = 2$: every interior and downstream-edge segment moves fully to the feasible side of the line. Segments $k = 2$ through $k = 6$ collapse to near-zero failure. The downstream edge is resolved, not merely improved.



The figure also makes plain what this lever cannot do. With the downstream edge resolved, the entry edge stands alone: $k = 0$ still fails on every input and $k = 1$ on nearly all. These segments inherit their disturbance from the arrival process, not the slot schedule, and no choice of η_r reaches them. The entry edge is the subject of the next chapter.

8. The Entry Edge

The exit edge and the entry edge are mirror images. Both are segments pinned to a boundary condition imposed from outside the buffer, and both pay a margin deficit for it. The difference is what they are pinned to. The exit edge is pinned to a constant — the handoff velocity v_d — so its deficit was deterministic and a single rate parameter resolved it. The entry edge is pinned to the arrival process, and the arrival process is not a constant: it has a mean and it has a variance, and the entry edge inherits both. This gives the entry edge two distinct problems where the exit edge had one.

The first is the mean. Segment $k = 0$ receives each input at the spacing the upstream delivers, and the mean of that spacing, μ_u , is small relative to what the early segments need. When inputs arrive too close together, segment $k = 0$ has too little free window between clearing input i and receiving input $i + 1$: $W_{i,0}$ is small, frequently negative, and the margin $M_{i,0}$ follows it across zero — as a synchronization shortfall when the window is merely too short, as outright co-occupancy when $W_{i,0}$ itself is negative. This is the same kind of deficit the exit edge had — too little room — and it admits the same kind of fix. The second problem is the variance: the

arrival process scatters inputs around the mean, and that scatter does not vanish when the mean is corrected. We treat the mean first, because it is the larger and simpler deficit, and return to the variance once the mean is in hand.

We solved the exit edge by increasing the pitch — widening the spacing between inputs so that each pinned transition had more time. The entry edge takes the same approach on the other side of the buffer: increase the pitch entering it. The spacing into the buffer is set by the velocity of whatever conveys the inputs there, and the natural lever is to convey them faster. A constant-velocity belt running faster than its infeed stretches the spacing of everything riding on it, by exactly the velocity ratio. Insert such a belt between the source and the buffer — the conditioning conveyor — running at velocity $v_c > v_u$, and the mean pitch arriving at the buffer becomes

$$\mu_b = \frac{v_c}{v_u} \mu_u > \mu_u.$$

More mean spacing at the buffer entry is more free window for the early segments, which is exactly the resource $k = 0$ and $k = 1$ were short of. The mean problem yields to the same mechanism as the exit edge: more pitch, more time, more margin.

The conditioning conveyor's velocity sits between two speeds the system already fixes. The source runs at v_u and the downstream at v_d , and the belt must run somewhere between them. We parameterize where with a single scalar $\eta_v \in [0, 1]$ — the fraction of the source-to-downstream velocity gap that the conditioning conveyor closes:

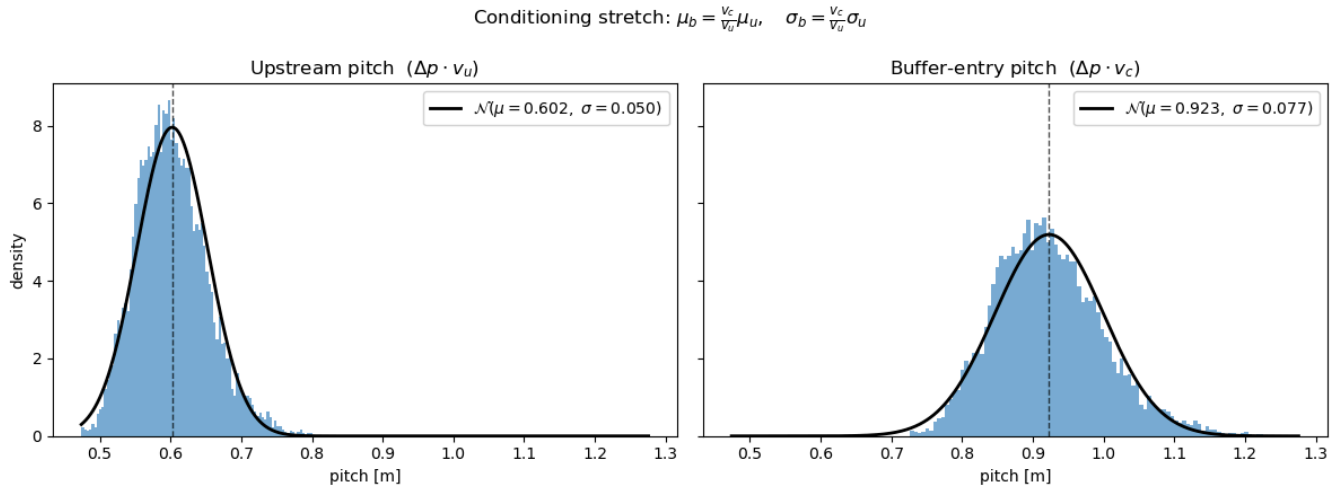
$$v_c = (1 - \eta_v) v_u + \eta_v v_d.$$

At $\eta_v = 0$ the belt runs at v_u and does no stretching; at $\eta_v = 1$ it runs at the full v_d and stretches maximally. The stretch ratio v_c/v_u ranges correspondingly from 1 to v_d/v_u , so η_v sets both the conditioning velocity and the amount of pitch stretch in a single number. It is the entry edge's lever, the counterpart to the rate surplus that resolved the exit edge.

The stretch does not act on the mean alone. The conditioning belt applies the same velocity ratio to every input, so it scales the entire pitch distribution — and scaling a distribution scales its standard deviation by the same factor. For the Gaussian arrival process this is immediate: a linear scaling of a normal variable multiplies its standard deviation by the scaling constant. The spread of the pitch arriving at the buffer is therefore

$$\sigma_b = \frac{v_c}{v_u} \sigma_u,$$

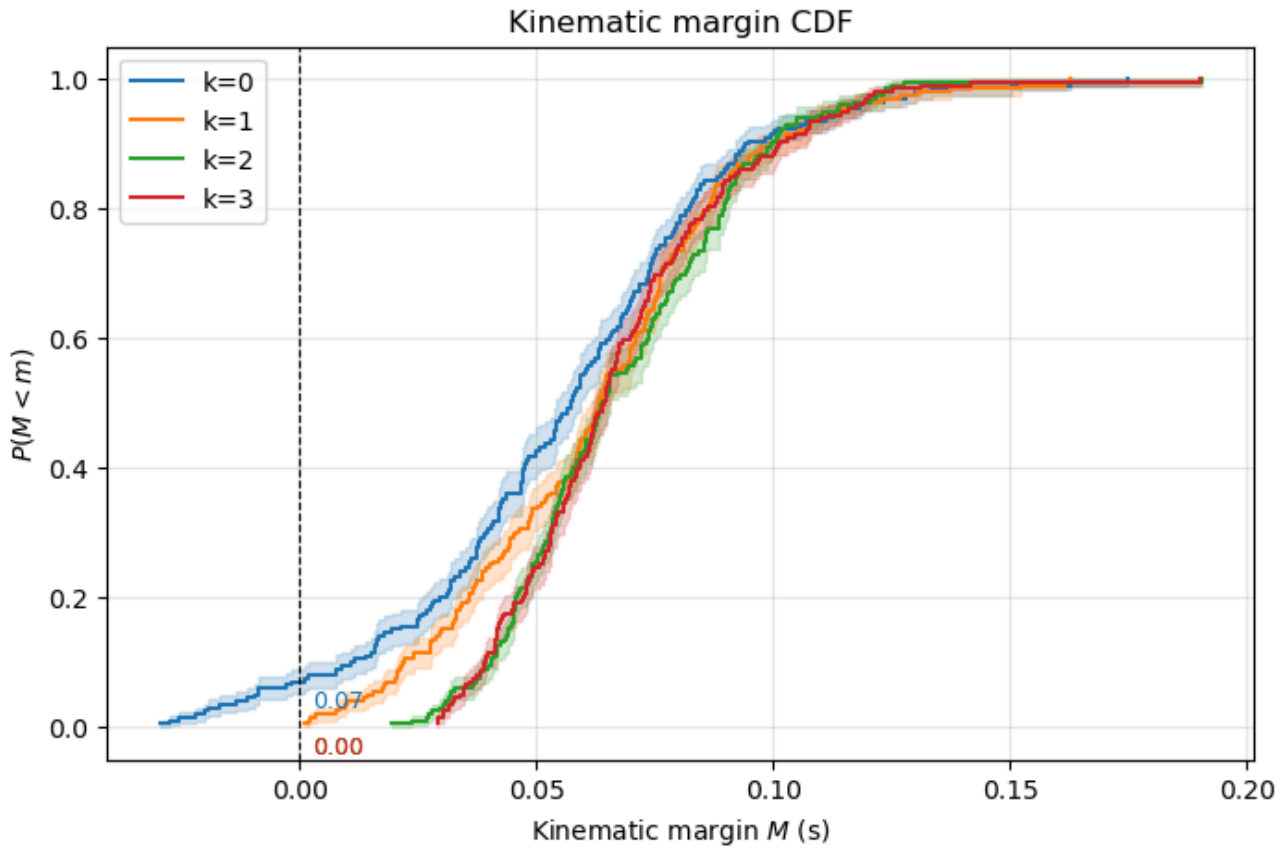
the same ratio that widened the mean. A direct numerical check confirms that both μ_b and σ_b track v_c/v_u , but it needs no derivation — it is a property of the linear map.



The conditioning conveyor runs at $v_c = 2.7576$ against a source velocity of $v_u = 1.8$, a stretch ratio of $v_c/v_u \approx 1.532$. Applied to the source pitch ($\mu_u = 0.6$, $\sigma_u = 0.05$), this predicts a buffer-entry pitch with mean $1.532 \times 0.6 \approx 0.92$ and standard deviation $1.532 \times 0.05 \approx 0.077$. The fitted distributions read $\mu_b = 0.923$ and $\sigma_b = 0.077$ — both scaled by the same ratio, confirming that the conditioning stretch acts as a single linear map: it carries the mean and the standard deviation up together, and the wider mean spacing cannot be bought without the proportionally wider spread. The fitted Gaussians sit slightly high and to the left of each histogram peak because the underlying pitch is generated by the log-space AR(1) process and is therefore mildly right-skewed rather than exactly normal; the Gaussian is an approximation to a slightly skewed marginal, which does not affect the linear scaling — the same ratio carries mean and standard deviation regardless of the exact shape of the distribution.

So the lever that buys mean spacing for the entry edge necessarily inflates the variability that arrives there. The two move together, locked by the single ratio v_c/v_u : there is no setting of η_v that delivers more mean room without also delivering more spread.

The effect on the entry-edge margin is visible in its distribution. The figure below shows the CDF of the entry-edge margin $M_{i,0}$ with the conditioning stretch applied.



The bulk of the distribution has moved to the feasible side of zero: the chronic, every-input failure that $k = 0$ suffered without the stretch is gone, because the wider mean spacing now gives the early segments room. What remains is a thin negative tail — the inputs that still arrive too tightly spaced despite the wider mean. These are the low end of the now-wider σ_b , the inflated variance reaching across zero on its unlucky draws. The mean problem is solved; this tail is the variance problem.

The obvious response is to push η_v higher still, buying yet more mean spacing to drown the tail. This does not work, and the reason is the coupling just established: raising η_v widens σ_b in lockstep with μ_b , so the tail it is trying to eliminate widens as fast as the bulk shifts away from it. Beyond a point, more stretch feeds the tail rather than starving it. The entry edge cannot be closed by mean spacing alone, and η_v cannot be pushed as far as we would like.

The residual tail has a structure of its own. It is the signature of inputs placed in consecutive slots — and that phenomenon is not confined to the entry edge. It is a property of the whole system and a direct consequence of having solved both edges together, and it is the subject of the next chapter.

9. Consecutive Placement

Solving both edges leaves one residual, and it belongs to neither edge alone. Take the system with both fixes in place — the conditioning stretch widening the entry pitch, and a generous rate surplus $\eta_r = 2$ holding the exit edge open. By the exit-edge chapter this should be comfortable:

twice as many slots as inputs, ample room to leave gaps, the downstream-edge segments fully feasible. Yet run a long batch and small pitches still appear at the exit segments, and with them, errors. They are infrequent, but they are there, and no further increase in η_r removes them cleanly. The effect survives both edge fixes because it is not an edge effect — it is a property of the whole pipeline, and it is the direct consequence of having solved the two edges together.

The signature of the effect is a pair of inputs placed in consecutive slots. Inputs i and $i + 1$ are assigned to slots j and $j + 1$ — adjacent indices, no slot skipped between them. Under a rate surplus this is the exception rather than the rule: with $r_d > r_u$ the greedy assignment usually leaves one or more empty slots between consecutive inputs, and a consecutive placement is what happens when it cannot. Understanding why it cannot, and why the resulting pair produces small exit pitch, is the content of this chapter.

The mechanism runs through the time horizon and the greedy assignment together. Recall from the drift chapter that the time horizon H_i — the gap between an input's arrival and its assigned slot — drifts upward under the Lindley recursion: the reflecting floor at $T_{\min}$ truncates the downward steps and lets the upward ones accumulate, so H_i trends up across the batch and is pulled back down only by the rate surplus and the skips it forces. When H_i is large, input i has a long time horizon and must therefore travel slowly across the buffer — a long horizon is a slow trajectory, a low exit velocity relative to the slot grid. Now consider input $i + 1$. The greedy assignment gives it the *earliest* feasible slot, by construction; it does not look ahead or reserve room. If input i has drifted to a late slot, the earliest slot still available to $i + 1$ may be the very next one, $j + 1$. But that slot arrives soon — input $i + 1$ has a *small* time horizon, because its slot is close — and a small horizon demands a fast trajectory. Input $i + 1$ must accelerate to reach $j + 1$ in time.

The two inputs therefore cross the buffer at opposite tendencies: input i slow, input $i + 1$ fast, the two driven apart in velocity by the horizon gap between them. At the exit segments this velocity difference becomes a spacing difference. Input $i + 1$, travelling faster, closes on the slow input i ahead of it, and the pitch between them at the handoff collapses. A small exit pitch is exactly the condition the exit-edge segments cannot absorb — too little free window between handing off input i and receiving input $i + 1$ — and the margin goes negative. The consecutive placement is the cause; the small exit pitch and its error are the effect.

There is a way to see what the buffer is doing here. The arrival variance is spread evenly across the input stream — every inter-arrival gap carries a little of it, independently. The buffer, in synchronizing that stream to a fixed grid, does not dissipate the variance; it relocates it. The accumulated effect of many small, evenly-spread upstream fluctuations is concentrated into the occasional tightly-spaced pair at the exit. The buffer translates variance that was distributed *between* inputs into discrete *events* at the end of the line. This is why the exit edge looked nearly variance-free in the exit-edge chapter and the residual here is sparse and bursty rather than smooth: the variance has not gone anywhere, it has been gathered into rare events.

The natural impulse is to forbid the configuration outright — to require that the assignment never fill two consecutive slots, guaranteeing a gap after every input. This cannot be done, for a reason already established. Forcing a gap after every placement imposes a hard floor on the inter-placement spacing, applied at every step — which is precisely the reflecting-barrier structure of the Lindley recursion. A hard minimum spacing enforced every input reintroduces the one-sided accumulation, and the time horizon drifts to infinity again, undoing the very thing the rate surplus was introduced to prevent. Non-consecutive placement cannot be made a constraint without resurrecting unbounded drift. The system must tolerate consecutive placement and manage its consequences; it cannot legislate the configuration away.

Within that tolerance, several directions are available. We name them as candidates and do not pursue them here.

The first is to push the rate surplus further — raise η_r beyond 2, presenting still more slots so that the earliest feasible slot for input $i + 1$ is less often the one immediately after input i . This thins the consecutive-placement rate, but only so far. A higher slot rate raises the downstream velocity $v_d = r_d d_s$, and the buffer must still reach that handoff velocity, so the surplus runs into the mechanical and kinematic speed limits of the system. There is a ceiling on η_r , and it arrives before the residual is gone.

The second is to shorten the buffer segments. Smaller segments need smaller free windows to stay feasible, so a given tight pair is more likely to clear $M_{i,k} \geq 0$ at the exit. This too can be pushed only so far: segments cannot be made arbitrarily short without the per-segment kinematics and the handoff overheads becoming the binding constraint, and the geometry has its own limits.

The third is to add a variable-length section after the buffer — a conveyor whose length can change, extending and retracting over the downstream carrier. By changing its length it shifts where and when the handoff occurs, which widens the set of time horizons available to each input: input $i + 1$ is no longer forced onto the single earliest slot the fixed grid offers, because the handoff point itself can move to meet a later one. In the model this would appear as a trajectory whose length is itself a control variable, rather than the fixed buffer length assumed throughout. This is the most capable of the three directions and the furthest from the current architecture, and modelling it properly is outside the scope of this project for now. We record it as a deliberate direction — considered, not pursued — and leave it for future work.

All three act on the same residual from different angles — the rate surplus and the geometry reduce the rate and the cost of the tight pair, the variable length section removes the constraint that forces it — and a complete solution most likely combines them. We characterize consecutive placement to this point and leave its resolution as future work.